

mvn11mi45

January 5, 2025

0.1 Problema 2:

0.1.1 Este exercício é dirigido à prova de correção do algoritmo estendido de Euclides apresentado no trabalho TP3

- a) Construa a asserção lógica que representa a pós-condição do algoritmo. Note que a definição da função

gcd

é

$$\text{gcd}(a, b) \equiv \min\{r > 0 \mid \exists s, t. r = a * s + b * t\}$$

.

```
[7]: from z3 import *

def verifica_pos_condicao(a_val, b_val, r_val, s_val, t_val):
    # Criar solver
    solver = Solver()

    # Declarar variáveis
    a = Int('a')
    b = Int('b')
    r = Int('r')
    s = Int('s')
    t = Int('t')

    # Adicionar valores concretos
    solver.add(a == a_val)
    solver.add(b == b_val)
    solver.add(r == r_val)
    solver.add(s == s_val)
    solver.add(t == t_val)

    # Pós-condição
    pos_cond = And(
        r > 0, # r é positivo
        r == a * s + b * t, # r é combinação linear de a e b
        ForAll([Int('x'), Int('y')], # Para todo x,y
            Implies(
                And(
```

```

        Int('x') == a * Int('y') + b * Int('y'),
        Int('x') > 0
    ),
    Int('x') >= r      # r é o menor valor positivo
)
)
)

solver.add(pos_cond)

# Verificar
if solver.check() == sat:
    modelo = solver.model()
    return True, f"Pós-condição satisfeita: {r_val} = {a_val}*{s_val} + {b_val}*{t_val}"
else:
    return False, "Pós-condição não satisfeita"

def teste():
    casos_teste = [
        (48, 36, 12, 1, -1),      # Caso válido
        (17, 5, 1, -2, 7),       # Caso válido
        (48, 36, 12, -1, 2)      # Caso inválido
    ]

    for a, b, r, s, t in casos_teste:
        print(f"\nTestando a={a}, b={b}, r={r}, s={s}, t={t}")
        sucesso, mensagem = verifica_pos_condicao(a, b, r, s, t)
        print(f"Resultado: {'Sucesso' if sucesso else 'Falha'}")
        print(f"Mensagem: {mensagem}")

if __name__ == "__main__":
    print("Testando Pós-condição com Z3:")
    teste()

```

Testando Pós-condição com Z3:

Testando a=48, b=36, r=12, s=1, t=-1

Resultado: Sucesso

Mensagem: Pós-condição satisfeita: 12 = 48*1 + 36*-1

Testando a=17, b=5, r=1, s=-2, t=7

Resultado: Sucesso

Mensagem: Pós-condição satisfeita: 1 = 17*-2 + 5*7

Testando a=48, b=36, r=12, s=-1, t=2

Resultado: Falha

Mensagem: Pós-condição não satisfeita

Para a realização deste exercício utilizamos a biblioteca `z3` como auxílio. Definimos a função `verifica_pos_condicao()` que recebe 5 valores: `'a_val'`, `'b_val'`, `'r_val'`, `'s_val'` e `'t_val'`. Esta verifica se os valores satisfazem uma pós condição específica.

É criado o solver através da variável `solver = Solver()` que é responsável por resolver o problema dado.

São definidas as variáveis do tipo “Int”. E fazemos atribuições dos valores fornecidos às variáveis, que permite que o solver use estes mesmos valores no processo de verificação.

De seguida definimos a pós-condição que é composta por três condições:

1. $r > 0$:

A condição exige que o valor de `r` seja sempre positivo, caso contrário a mesma não vai funcionar.

2. Combinação linear das variáveis `a` e `b`:

0.2 $r = a s + b t$

Esta condição expressa a variável `r` como uma combinação linear das variáveis `a` e `b` com os coef.

3. Condição sobre `x` e `y`:

Para todos os inteiros `x` e `y`, se a condição:

0.3 $x = a y + b y > 0$

for satisfeita então garantimos que:

0.4 $x \geq r$

Esta condição exige que qualquer valor `x` que resulte da combinação linear de `a` e `b` é maior ou

4. Verificação da pós-condição:

Depois de adicionar a pós-condição ao solver, o código vai verificar se o problema é `sat` ou `unsat` (satisfazível ou insatisfazível). Caso seja satisfazível o programa retorna o valor booleano `True` com uma mensagem a dizer que a pós-condição foi satisfeita. Caso contrário é devolvido o valor booleano `False` com uma mensagem a dizer que a pós-condição não foi satisfeita.

Para finalizar a alínea a) fizemos um programa simples que testa diferentes conjuntos de valores.

- b) Usando a metodologia do comando `havoc` para o ciclo, escreva o programa na linguagem dos comandos anotados (LPA). Codifique a pós-condição do algoritmo com um comando `assert`.

```
[15]: from z3 import *

def lpa(a_val, b_val):
    solver = Solver()
```

```

# Criar variáveis simbólicas para o estado inicial
a = Int('a')
b = Int('b')
r = Int('r')
r_linha = Int('r_linha')
s = Int('s')
s_linha = Int('s_linha')
t = Int('t')
t_linha = Int('t_linha')
q = Int('q')

# Estado inicial
solver.add(a == a_val)
solver.add(b == b_val)
solver.add(r == a_val)
solver.add(r_linha == b_val)
solver.add(s == 1)
solver.add(s_linha == 0)
solver.add(t == 0)
solver.add(t_linha == 1)

# Invariante inicial
solver.add(r == a * s + b * t)
solver.add(r_linha == a * s_linha + b * t_linha)

estados = []
r_atual, r_linha_atual = a_val, b_val
s_atual, s_linha_atual = 1, 0
t_atual, t_linha_atual = 0, 1

while r_linha_atual != 0:
    q = r_atual // r_linha_atual

    # Criar novas variáveis para o próximo estado
    r_prox = Int(f'r_{len(estados)}')
    r_linha_prox = Int(f'r_linha_{len(estados)}')
    s_prox = Int(f's_{len(estados)}')
    s_linha_prox = Int(f's_linha_{len(estados)}')
    t_prox = Int(f't_{len(estados)}')
    t_linha_prox = Int(f't_linha_{len(estados)}')

    # Adicionar restrições de transição
    solver.add(r_prox == r_linha_atual)
    solver.add(r_linha_prox == r_atual - q * r_linha_atual)
    solver.add(s_prox == s_linha_atual)
    solver.add(s_linha_prox == s_atual - q * s_linha_atual)
    solver.add(t_prox == t_linha_atual)

```

```

solver.add(t_linha_prox == t_atual - q * t_linha_atual)

# Atualizar valores atuais
r_atual, r_linha_atual = r_linha_atual, r_atual - q * r_linha_atual
s_atual, s_linha_atual = s_linha_atual, s_atual - q * s_linha_atual
t_atual, t_linha_atual = t_linha_atual, t_atual - q * t_linha_atual

estados.append((r_atual, r_linha_atual, s_atual, s_linha_atual,
↪t_atual, t_linha_atual, q))

# Verificar se o modelo é satisfatível
if solver.check() == sat:
    return True, estados
else:
    return False, "Não foi possível encontrar uma solução válida"

def testar_lpa():
    casos_teste = [
        (48, 36),
        (17, 5),
        (21, 14)
    ]

    for a, b in casos_teste:
        print(f"\nExecutando programa LPA para a={a}, b={b}")
        sucesso, resultado = lpa(a, b)

        if sucesso:
            print("Estados gerados:")
            for i, estado in enumerate(resultado):
                r, r_linha, s, s_linha, t, t_linha, q = estado
                print(f"Estado {i}: r={r}, r'={r_linha}, s={s}, s'={s_linha},
↪t={t}, t'={t_linha}, q={q}")
            print(f"MDC encontrado: {resultado[-1][0]}")
        else:
            print(f"Erro: {resultado}")

if __name__ == "__main__":
    print("Teste do programa LPA:")
    testar_lpa()

```

Teste do programa LPA:

Executando programa LPA para a=48, b=36

Estados gerados:

Estado 0: r=36, r'=12, s=0, s'=1, t=1, t'=-1, q=1

Estado 1: r=12, r'=0, s=1, s'=-3, t=-1, t'=4, q=3

MDC encontrado: 12

Executando programa LPA para a=17, b=5

Estados gerados:

Estado 0: r=5, r'=2, s=0, s'=1, t=1, t'=-3, q=3

Estado 1: r=2, r'=1, s=1, s'=-2, t=-3, t'=7, q=2

Estado 2: r=1, r'=0, s=-2, s'=5, t=7, t'=-17, q=2

MDC encontrado: 1

Executando programa LPA para a=21, b=14

Estados gerados:

Estado 0: r=14, r'=7, s=0, s'=1, t=1, t'=-1, q=1

Estado 1: r=7, r'=0, s=1, s'=-2, t=-1, t'=3, q=2

MDC encontrado: 7

notebook controller is DISPOSED.

View Jupyter log for further details.

Para esta alínea implementamos o Algoritmo de Euclides estendido utilizando o solver da biblioteca **z3**. Este programa calcula o máximo divisor comum ou MDC de dois números inteiros e em simultâneo determina os coeficientes da combinação linear que resulta no MDC.

Definimos primeiramente a função do programa, **lpa()** que recebe os valores **a_val** e **b_val**. São definidas as seguintes variáveis: **a**, **b**, **r**, **s**, **t**, **q**. Definimos também o solver que é caracterizado com estado inicial, onde:

- **r == a_val**;
- **r_linha == b_val**;
- **s**, **s_linha**, **t** e **t_linha** recebem valores iniciais que garantem as combinações lineares.

São adicionadas ao solver as restrições que garantem a validade das combinações lineares, ou seja, são adicionadas os seguintes invariantes:

- **r = a * s + b * t**
- **r_linha = a * s_linha + b * t_linha**

Para a iteração dos estados utilizamos um *while loop* que é um ciclo que permite calcular os próximos estados até que o valor de **r_linha**(que é o resto) seja zero. É calculado o quociente **q**, define-se o próximo estado com novas variáveis e por fim é atualizado os valores atuais com base nas transições.

Para o próximo passo verificamos se o modelo é satisfazível, assim como na alínea anterior se for **sat** é satisfazível e retorna os estados gerados, caso contrário é **unsat** insatisfazível e devolve uma mensagem a dizer que não é possível encontrar uma solução válida.

Para finalizar esta alínea, assim como na alínea o grupo fez uma programa que testa com um conjunto de valores a viabilidade do programa realizado.

- c) Construa codificações do programa LPA através de transformadores de predicados “strongest post-condition” e prove a correção do programa LPA.

```

[ ]: from z3 import *

# Definição do Strongest Post-Condition (SP) para o programa LPA
# O programa LPA resolve o problema do algoritmo estendido de Euclides

# Inicialização das variáveis e condições iniciais
a, b = Ints('a b') # Entradas
r, r_linha = Ints('r r_linha') # Restos
s, s_linha = Ints('s s_linha') # Coeficientes de a
t, t_linha = Ints('t t_linha') # Coeficientes de b
q = Int('q') # Quociente

# Condições iniciais
init = And(
    a > 0, b > 0, # Pré-condição: a e b são positivos
    r == a, r_linha == b, # Restos iniciais
    s == 1, s_linha == 0, # Coeficientes iniciais de a
    t == 0, t_linha == 1 # Coeficientes iniciais de b
)

# Invariante do algoritmo (mantém-se verdadeiro após cada iteração)
invariante = And(
    r == a * s + b * t,
    r_linha == a * s_linha + b * t_linha,
    r_linha >= 0 # r_linha não pode ser negativo
)

# Corpo do laço (iteração do algoritmo)
loop = And(
    r_linha != 0, # O loop continua enquanto r_linha != 0
    q == r / r_linha, # Quociente q
    r == r_linha, # Atualização de r
    r_linha == r - q * r_linha, # Atualização de r'
    s == s_linha, # Atualização de s
    s_linha == s - q * s_linha, # Atualização de s'
    t == t_linha, # Atualização de t
    t_linha == t - q * t_linha # Atualização de t'
)

# Pós-condição (após o término do programa)
pos = And(
    r_linha == 0, # O último r_linha deve ser zero
    r > 0, # r contém o gcd(a, b)
    r == a * s + b * t # r é uma combinação linear de a e b
)

# Construção do SP

```

```

sp = And(init, Exists([r, r_linha, s, s_linha, t, t_linha, q], And(invariante,
↪loop, pos)))

# Prova de correção
solver = Solver()
solver.add(Implies(init, sp)) # Se o estado inicial é válido, então o SP deve
↪ser válido

if solver.check() == sat:
    print("Correção provada com sucesso!")
else:
    print("A correção falhou.")

```

Correção provada com sucesso!

notebook controller is DISPOSED.

View Jupyter [<log>](command:jupyter.viewOutput) for further details.

1 Objétivo:

O objétivo desta alínea é a construção da codificação do programa LPA(Linear Program Analysis) com a utilização de transformadores de perdicados conhecidos como *Strongest Pos-Condition*(SP). Este processo permite-nos realizar diferentes funções, como: a formalização do comportamento do programa através de lógica matemática. Permite fazer provas de correção do algoritmo LPA nas suas condições iniciais, invariantes e pós-condições e consegue demonstrar que o programa é consistente com as suas especificações estabelecidas.

O algoritmo em questão é a implementação do método estendido de Euclides, que é usado para encontrar o maior divisor comum(mdc) de dois números inteiros e permite expressá-lo como uma combinação linear desses mesmos números.

2 Implementação:

1. Variáveis:

As variáveis definidas neste programa foram definidas como “Entradas”(inteiros e números positivos), “Restos”(restos durante as iterações cálculos), “Coeficientes”(são os coeficientes que vão satisfazer a combinação linear), “Quoeficiente”(quoeficiente calculado em cada iteração).

2. Condições Iniciais(Pré-condição):

A condição inicial definida na variável “init” estabelece os valores de algumas variáveis iniciais, como:

- $a > 0, b > 0$: As variáveis ‘a’ e ‘b’ têm que ser sempre positivas;
- $r == a, r_linha == b$: define que as variáveis ‘r’ e ‘r_linha’ são definidas como restos iniciais.

- $s == 1, s_linha == 0$: Define os coeficientes iniciais de a.
- $t == 0, t_linha == 1$: Define os coeficientes iniciais de b.

3. Invariante do Algoritmo:

O invariante (uma expressão lógica que permanece sempre verdadeira após cada iteração) definido neste sistema:

```
invariante = And(
    r == a * s + b * t,
    r_linha == a * s_linha + b * t_linha,
    r_linha >= 0  # r_linha não pode ser negativo
)
```

, garante que as combinações lineares são sempre consistentes e que 'r_linha' nunca é negativo.

4. Loop de Iterações:

O loop definido codifica as iterações do algoritmo e a cada iteração o quociente é calculado e as variáveis utilizadas são atualizadas até verificar que 'r_linha' == 0, quando é verificado o loop é finalizado.

```
loop = And(
    r_linha != 0,  # O laço continua enquanto r_linha != 0
    q == r / r_linha,  # Quociente q
    r == r_linha,  # Atualização de r
    r_linha == r - q * r_linha,  # Atualização de r'
    s == s_linha,  # Atualização de s
    s_linha == s - q * s_linha,  # Atualização de s'
    t == t_linha,  # Atualização de t
    t_linha == t - q * t_linha  # Atualização de t'
)
```

5. Pós-condição:

A pós-condição retrata o estado final que é esperado do algoritmo, sendo este:

- O último resto é zero;
- O maior divisor comum é positivo;
- 'r' é uma combinação linear de 'a' e 'b';

```
pos = And(
    r_linha == 0,  # O último r_linha deve ser zero
    r > 0,  # r contém o mdc(a, b)
    r == a * s + b * t  # r é uma combinação linear de a e b
)
```

6. Construção do Strongest Post-Condition(SP):

O SP é definido como uma conjunção inicial, do invariante, do loop e da pós-condição. Este representa a correção do programa como um todo.

```
sp = And(init, Exists([r, r_linha, s, s_linha, t, t_linha, q], And(invariante, loop, pos)))
```

7. Prova da correção:

Para a prova da correção utilizamos, a partir da biblioteca z3, o SMT Solver. Isto é, para verificar se o SP é logicamente válido/implicado pela condição inicial.

```
solver = Solver()
solver.add(Implies(init, sp))

if solver.check() == sat:
    print("Correção provada com sucesso!")
else:
    print("A correção falhou.")
```

2.0.1 Conclusão:

Ao executar o código realizado, o SMT Solver devolveu a frase “Correção aprovada com sucesso”, o que indica que o programa LPA está consistente com as suas especificações, ou seja, as condições iniciais garantem a funcionalidade correta do algoritmo, o invariante permanesse constante/preservado em cada iteração feita e o algoritmo termina em um estado onde a pós-condição definida é satisfeita.